MPMC — Module 2 — EXAM-READY NOTES

8086 Microprocessor: Architecture, Pin Diagram, Addressing Modes, Instruction Set, Assembler Directives & Interrupts

Scope: These notes cover **Module 2 only** (ECE3004 – Microprocessors and Microcontrollers). Every slide topic from the Module 2 deck is included: generations of microprocessors, 8086 overview, pins & signals (common/minimum/maximum mode), BIU–EU architecture, registers, flags, physical address calculation, all 12 addressing modes, all 6 instruction groups, string instructions, branch/loop instructions, assembler directives, procedures vs macros, modular programming and 8086 interrupts (IVT, interrupt processing, interrupt types).

This master file is also split into 3 topic-wise files (same content, easier to study/reprint):

- **Part 1 — Architecture** → `MPMC_Module2_Part1_Architecture.md`
- **Part 2 — Addressing Modes** → `MPMC_Module2_Part2_Addressing_Modes.md`
- **Part 3 — Instruction Set, Directives & Programming** → `MPMC_Module2_Part3_Instruction_Set.md`

1. GENERATIONS OF MICROPROCESSORS

Generation	Period	Technology	Word size / Pins	Key features	Example
First	1971 – 1973	**PMOS** (not TTL compatible)	4-bit → 16 pins; 8- & 16-bit → 40 pins	Signals had to be **multiplexed** due to pin limitations	Intel 4004/4040
Second	1973 onward	**NMOS** (faster, higher density, TTL compatible)	4 / 8 / 16-bit → 40 pins	Large memory & I/O addressing, greater subroutine nesting, better interrupt handling	**Intel 8085 (8-bit)**
Third	1978 onward	**HMOS** (faster, higher packing density)	16-bit → 40 / 48 / 64 pins	Easier to program, dynamically relocatable programs, **multiply/divide hardware**, powerful interrupts, flexible I/O addressing	**Intel 8086 (16-bit)**
Fourth	1980s	**HCMOS** (low-power HMOS)	32-bit	Physical memory 2^{24} = **16 MB**, virtual memory 2^{40} = **1 TB**, **floating-point hardware**, many addressing modes	Intel 80386
Fifth	—	—	32/64-bit	Advanced pipelining, superscalar	**Pentium**

Memory trick: P-N-H-H-P = PMOS → NMOS → HMOS → HCMOS → Pentium.

2. GENERAL MICROPROCESSOR FUNCTIONAL BLOCKS

The general block diagram of any microprocessor has:

Block	Function
ALU (Computational Unit)	Performs arithmetic and logic operations
Flag Register	Stores various conditions of the result as status bits (flags)
Register array / internal memory	Internal storage of data
PC / IP	Generates the address of the **next instruction** to be fetched from memory, sent via the **address bus**
Instruction decoding unit	Decodes instructions; sends information to the timing and control unit
Timing and control unit	Generates control signals for internal and external operations of the μ P

Buses: Control bus, **Address bus**, **Data bus** connect these blocks to memory and I/O.

3. 8086 MICROPROCESSOR — OVERVIEW & SALIENT FEATURES

- **First 16-bit processor** released by **INTEL in 1978**.
- Originally **HMOS**; now manufactured using **HMOS III** technique.
- **≈29,000 transistors**, **40-pin DIP**, **+5 V** supply.
- **No internal clock generator** → needs an **external asymmetric clock** with **33 % duty cycle** (from the **8284A** clock generator).
- **20-bit address bus** → can address **2^{20} = 1 MB (1 megabyte)** of memory.
- 16-bit data bus → word size is 16 bits.
- Works in two modes: **Minimum mode** (single processor) and **Maximum mode** (multiprocessor/coprocessor).

Quick comparison (Module-2 favourites):

Feature	8085	8086
Word length	8-bit	16-bit
Address bus	16-bit	20-bit
Memory	64 KB	1 MB
Clock	3 MHz	5 / 8 / 10 MHz (varies)
Multiplexing	Lower address/data	Address/data + status multiplexed
Pipelining	No	**Yes (6-byte queue)**

4. 8086 PIN DIAGRAM AND SIGNALS (40-pin DIP)

Pin layout (memorise the groups, not all 40 numbers):

Pins	Name
1, 20	GND (ground)
2–16, 39	**AD0–AD15** (address/data bus, bidirectional, multiplexed)
35–38	**A16/S3, A17/S4, A18/S5, A19/S6**
17	**NMI**
18	**INTR**
19	**CLK**
21	**RESET**
22	**READY**
23	**TEST**
24	**QS1**, 25 = **QS0** (max mode) / 25 = **ALE** (min mode)
26	**DEN** (min) / S0 (max)
27	**DT/R** (min) / S1 (max)
28	**M/IO** (min) / S2 (max)
29	**WR** (min) / LOCK (max)
30	**HLDA** (min) / RQ/GT1 (max)

Pins	Name
31	**HOLD** (min) / RQ/GT0 (max)
32	**RD**
33	**MN/MX** (minimum/maximum mode select)
34	**BHE/S7**
40	VCC

Exam tip: The same pin carries **different names in minimum and maximum modes** — this is the single most repeated pin question.

4.1 Common Signals (both modes)

Signal	Direction	Description
AD0–AD15	Bidirectional, multiplexed	Low-order address bus multiplexed with data. Written **A0–A15** when carrying addresses, **D0–D15** when carrying data.
A16/S3 – A19/S6	Output, multiplexed	High-order address bus multiplexed with **status signals**.
BHE/S7 (active low)	Output	**Bus High Enable**: enables data onto the most significant half of the data bus (**D8–D15**). 8-bit devices connected to the upper half of the data bus use BHE. Multiplexed with status S7.
MN/MX	Input	**1 → Minimum mode; 0 → Maximum mode** (indicates what mode the processor is to operate in).
RD (active low)	Output	Read operation; active low.
READY	Input, active HIGH	Acknowledgement from a **slow device/memory** that it has completed the data transfer. Synchronised by the **8284A clock generator** before being given to the 8086.
RESET	Input, active HIGH	Immediately terminates the present activity; must be **active HIGH for at least 4 clock cycles**.
CLK	Input	Basic timing for processor operation and bus control activity; an **asymmetric square wave with 33 % duty cycle**.
INTR	Input, active HIGH, level triggered	**Interrupt Request**, sampled during the **last clock cycles of each instruction** to determine availability of the request. If pending → processor enters the **interrupt acknowledge cycle**. Internally synchronised.
NMI	Input	Non-maskable interrupt (positive edge triggered).
TEST	Input	Used with WAIT; if TEST = 0 the processor continues, else waits.

Note: The 8086 does **not** have an on-chip clock generation circuit → the **8284 clock generator** is connected to the CLK pin.

4.2 Minimum-Mode Signals (MN/MX = 1)

Signal	Description
ALE (Address Latch Enable)	Demultiplexes the address/data lines using **external latches (8282)**. A positive pulse is generated each time the processor begins any operation; indicates a valid address on the address/data lines. Also differentiates memory access from I/O access (high for memory reference, low for IN/OUT).
DEN (Data Enable)	Output enable for the **8286 transceiver**. Also called Data Enable; enables the transceiver that separates data from the address/data bus.
DT/R (Data Transmit/Receive)	Controls the **direction of data flow** through the data transceivers.
M/IO	Used to differentiate memory access and I/O access: **high for memory reference**, **low for IN and OUT** instructions.
WR (active low)	Write control signal; asserted low whenever the processor writes data to memory or I/O port.
INTA (active low)	**Interrupt Acknowledge**: becomes low on this line when the interrupt request is accepted by the processor.
HOLD	Input from a bus master requesting **control of the bus** (usually by the **DMA controller**).
HLDA (Hold Acknowledge)	Acknowledge by the processor to the bus master via HOLD; asserted **high** when the processor accepts HOLD.

Demultiplexing hardware (very common exam figure): 8282 latch (strobed by ALE) separates **A0–A19** (and gives BHE), while the **8286 transceiver** (controlled by DT/R and DEN) handles **D0–D15**. That's how the multiplexed AD bus is converted into separate address and data buses.

4.3 Maximum-Mode Signals (MN/MX = 0)

Signal	Description
S0, S1, S2 (active low)	Status signals used by the **8288 bus controller** to generate bus timing and control signals.
QS0, QS1	**Queue Status**: tell the status of the instruction queue; external devices can track the internal queue status.
LOCK (active low)	Locks the bus during the next instruction; prevents other bus masters from taking the bus.
RQ/GT0, RQ/GT1 (Request/Grant)	Used to request and grant bus access by other processors (e.g., 8087 coprocessor) — bidirectional.

S2 S1 S0 → Machine Cycle (MUST MEMORISE):

S2	S1	S0	Machine Cycle
0	0	0	**Interrupt acknowledge**
0	0	1	Read I/O port
0	1	0	Write I/O port
0	1	1	**Halt**
1	0	0	**Code access** (instruction fetch)
1	0	1	Read memory
1	1	0	Write memory
1	1	1	Passive / Inactive

QS1 QS0 → Queue operation:

QS1	QS0	Queue operation
0	0	No operation
0	1	First byte of an opcode from queue
1	0	**Empty the queue**
1	1	Subsequent byte from queue

4.4 Minimum vs Maximum Mode

Point	Minimum Mode	Maximum Mode
MN/MX pin	High (1)	Low (0)
Coprocessor use	Not associated with any coprocessor	Works in **multiprocessor / coprocessor** configuration
Bus controller	Not needed (8086 gives all signals itself)	**8288 bus controller** required
Signals generated	8086 generates ALE, DEN, DT/R, M/IO, WR, INTA, HOLD, HLDA	8086 gives S0–S2, QS0–QS1, LOCK, RQ/GT0–RQ/GT1
Typical use	Single-processor small systems	Multiprocessor systems (e.g., 8086 + 8087 + 8089)

5. INSIDE THE 8086 — THE ARCHITECTURE (BIU + EU)

5.1 Pipelining

- **Two ways to make a CPU faster:**
  1. **Increase working frequency** — technology dependent.
  2. **Change the internal architecture of the CPU** ← what Intel did.
- **Pipelining = allowing the CPU to fetch and execute at the same time.**
- Intel implemented it by splitting the 8086 into **two independent sections that work simultaneously**:
  - **Execution Unit (EU)** — executes instructions previously fetched.
  - **Bus Interface Unit (BIU)** — accesses memory and peripherals.
- **Benefit:** instruction fetch is **overlapped with execution** → speed-up.

5.2 Bus Interface Unit (BIU)

Functions: fetches instructions, reads data from memory and I/O ports, writes data to memory and I/O ports.
Contains:

1. **Dedicated adder** to generate the **20-bit physical address**.
2. **Four 16-bit segment registers** — **CS, DS, SS, ES**.
3. **Instruction Pointer (IP)**.
4. **6-byte instruction queue** (FIFO).

Segment registers details:

Register	Points to	Offset used	Notes
CS (Code Segment)	Base/start of current **code** segment	**IP**	20-bit physical address = CS shifted 4 bits left + IP. All instructions of a program are relative to CS×16 plus the offset provided by IP.
DS (Data Segment)	Current **data** segment	**SI / DI / 16-bit displacement**	Operands for most instructions are fetched from this segment.
SS (Stack Segment)	Current **stack**	**SP** (PUSH/POP), **BP** (based addressing)	Physical stack address = SS×16 + SP (or + BP).
ES (Extra Segment)	**Extra** segment; data in excess of 64 K pointed by DS	**DI** (string destination)	**String instructions use ES:DI** for the destination address.

Memory segmentation facts:

- 8086's **1 MB memory is divided into segments of up to 64 KB each**.
- Programs obtain access to code and data by **changing the segment register contents** to point to the desired segment.
- At any particular time, the 8086 can **directly address 4 segments (256 KB within the 1 MB)**.

Instruction Queue:

- A group of **First-In-First-Out (FIFO)** registers holding **up to 6 bytes** of instruction code **pre-fetched** from memory ahead of time.
- Done to speed up execution by **overlapping instruction fetch with execution** — this mechanism is **pipelining**.

Instruction Pointer (IP):

- 16-bit; **always points to the next instruction to be executed** within the currently executing code segment.
- Contains the **16-bit offset address** of the next instruction code within the 64 KB code segment.
- Automatically **incremented** as execution proceeds.

5.3 Execution Unit (EU)

Functions: decodes and executes instructions. A **decoder** in the EU control system translates instructions.
Contains:

- **16-bit ALU** for arithmetic and logic operations.
- **Four general-purpose registers**: **AX, BX, CX, DX** (each 16-bit; splittable into two 8-bit halves).
- **Pointer registers**: **SP, BP**.
- **Index registers**: **SI, DI**.
- **Flag register**.

8-bit splitting rule: AX→(AH, AL), BX→(BH, BL), CX→(CH, CL), DX→(DH, DL). The **low-order byte** goes to AL/BL/CL/DL and the **high-order byte** to AH/BH/CH/DH.

Exam one-liner: BIU fetches, EU executes — and they work simultaneously → pipelining.

6. REGISTERS OF 8086 (WITH SPECIAL FUNCTIONS)

Sl.	Type	Width	Registers
1	General purpose	16-bit / 8-bit	**AX, BX, CX, DX / AL, AH, BL, BH, CL, CH, DL, DH**
2	Pointer	16-bit	**SP, BP**
3	Index	16-bit	**SI, DI**
4	Instruction Pointer	16-bit	**IP**
5	Segment	16-bit	**CS, DS, SS, ES**
6	Flag (PSW)	16-bit	**Flag register**

Register	Name	Special Function
AX	16-bit Accumulator	Stores 16-bit results of arithmetic & logic operations
AL	8-bit Accumulator	Stores 8-bit results of arithmetic & logic operations
BX	Base register	Holds **base value in base addressing mode** to access memory data. **Only general-purpose register whose contents can be used for addressing memory**; memory references using BX default to **DS**.
CX	Count register	Holds **count value** for **SHIFT, ROTATE and LOOP** instructions

Register	Name	Special Function
DX	Data register	Holds data for **multiplication and division** operations; also holds the **16-bit port address** for indirect I/O
SP	Stack Pointer	Holds the **offset address of the top of stack** memory
BP	Base Pointer	Holds base value in **based addressing using SS** to access data from stack memory
SI	Source Index	Holds index value of **source operand (data)** for string instructions
DI	Destination Index	Holds index value of **destination operand (data)** for string operations

Extra notes from slides:

- **AX/AL** are used by **I/O instructions** (IN/OUT) for 16/8-bit data, and by **MUL/DIV** instructions.
- **CX example:** `LOOP START` automatically **decrements CX by 1 without affecting flags** and checks if CX = 0. If zero, the 8086 executes the next instruction; otherwise it branches to the label START.
- **SI & DI** are used in **indexed addressing**; string instructions use **SI with DS** and **DI with ES** to distinguish source and destination addresses.

6.1 Flag Register (PSW) — bit by bit

Bit	Flag	Name	Set (=1) when…
0	**CF**	Carry Flag	There is a **carry out of MSB** in addition or a **borrow** in subtraction
2	**PF**	Parity Flag	Lower byte of result contains an **even number of 1's** (else 0)
4	**AF**	Auxiliary Carry Flag	There is a carry from the **lowest nibble (bit 3)** during addition, or borrow from bit 3 during subtraction
6	**ZF**	Zero Flag	Result of computation/comparison is **zero**
7	**SF**	Sign Flag	Result of computation is **negative**
8	**TF**	Trap Flag	Set → processor enters **single-step execution mode** (generates internal interrupts after each instruction)
9	**IF**	Interrupt Flag	Controls recognition of **external maskable interrupts**; clearing IF **disables** these interrupts
10	**DF**	Direction Flag	Used by **string instructions**: **DF = 0 → auto-incrementing** (lowest → highest address); **DF = 1 → auto-decrementing** (highest → lowest address)
11	**OF**	Overflow Flag	Result of a **signed operation is too large** to fit in the destination (more than 7 bits for 8-bit, more than 15 bits for 16-bit signed operations)

Bit positions (memorise the layout):

```
15 14 13 12 11 10  9  8  7  6  5  4  3  2  1  0
            OF  DF  IF TF SF ZF    AF    PF    CF
```

Note: Bits 1, 3, 5, 12, 13, 14, 15 are undefined/unused in the 8086.

7. PHYSICAL ADDRESS GENERATION (MEMORY ACCESS)

Why needed? 8086 has **20 address lines** (→ 2^{20} = 1 MB) but the **largest register is only 16 bits**.

Definitions:

- **Physical Address:** the actual address of a byte in memory, i.e. the value which goes out onto the **address bus** (20 bits).
- Memory addresses are represented as **Segment : Offset** — e.g. **89AB:F012**.

Formula:

Physical Address = (Segment register × 16_{10}) + Offset

= shift the segment register **one hex digit left (append 0)** and **add the offset**.

Worked example (from slides) — 89AB:F012

```
  Segment  89AB  →  89AB0    (paragraph to byte: 89AB × 10H = 89AB0)
  Offset   F012  →   F012    (offset is already in byte units)
                    -----
  Physical address   98AC2 H
```

(16 bytes of contiguous memory = one "paragraph".)

Typical segment : offset pairs used:

Access type	Segment	Offset
Instruction fetch	CS	IP
Data (stack ops)	SS	SP
Data (based)	SS	BP
Data (default)	DS	BX / SI / DI / disp
String source	DS	SI
String destination	**ES**	**DI**

Addressing-modes notation used in the slides (write these in the exam):

- **EA** = Effective Address (offset part), **BA** = Base Address (segment × 16_{10}), **MA** = Memory Address = BA + EA.

8. ADDRESSING MODES OF 8086

Definition: Every instruction of a program operates on data. **The different ways in which a source operand is denoted in an instruction are known as addressing modes.**

The 12 modes, grouped as in the slides:

Group	Modes
Group I — register & immediate data	1. Register, 2. Immediate
Group II — memory data	3. Direct, 4. Register Indirect, 5. Based, 6. Indexed, 7. Based-Index, 8. String
Group III — I/O ports	9. Direct I/O port, 10. Indirect I/O port
Group IV — relative	11. Relative
Group V — implied	12. Implied

8.1 Register Addressing

The instruction specifies the **name of the register** which holds the data.

```
MOV CL, DH     (CL) ← (DH)
ADD AX, BX     (AX) ← (AX) + (BX)
```

Content of source **is NOT cleared** after execution.

8.2 Immediate Addressing

An **8-bit or 16-bit data is specified as part of the instruction**.

```
MOV DL, 08H    (DL) ← 08H
SUB AX, 0A9FH  (AX) ← (AX) − 0A9FH
```

8.3 Direct Addressing (Group II)

The **effective address of the memory location** at which the data operand is stored is **given directly in the instruction** (16-bit displacement written directly).

```
MOV [2000H], 02H
MOV [0234H], 0234H
MOV BX, [1354H]      ; copy contents of memory location 1354H into BX
MOV [1234H], BX
MOV BL, [1354H]
MOV [0400H], CL
```

The square brackets around 1354H denote the **contents of the memory location**.

8.4 Register Indirect Addressing

The name of the register which **holds the EA** is specified. Registers used: **BX, BP, DI, SI**. **DS** is used for the base address by default.

```
MOV CX, [BX]
```

```
EA = (BX)          BA = (DS) × 16₁₀        MA = BA + EA
(CX) ← (MA)   or  (CL) ← (MA), (CH) ← (MA+1)
```

8.5 Based Addressing

BX or BP holds the base value, plus a **signed 8-bit or unsigned 16-bit displacement**. An **8-bit displacement is sign-extended to 16-bit** before adding.

- **BX with DS** → physical address from BX and DS.
- **BP with SS** → physical address from BP and SS.

```
MOV AX, [BX + 08H]
MOV [BX + 1234H], AX
MOV [BP + 21H], 9878H
```

```
0008H ← 08H (sign extended) ; EA = (BX) + 0008H ; BA = (DS) × 16₁₀ ; MA = BA + EA
(AX) ← (MA)   [ (AL) ← (MA), (AH) ← (MA+1) ]
```

8.6 Indexed Addressing

SI or DI holds an index value, plus a **signed 8-bit or unsigned 16-bit displacement** (added to get EA).

```
MOV CX, [SI + 0A2H]
```

```
FFA2H ← A2H (sign extended) ; EA = (SI) + FFA2H ; BA = (DS) × 16₁₀ ; MA = BA + EA
(CX) ← (MA)
```

8.7 Based-Index Addressing

EA = **base register (BX or BP) + index register (SI or DI) + displacement**.

```
MOV DX, [BX + SI + 0AH]
```

```
000AH ← 0AH (sign extended) ; EA = (BX) + (SI) + 000AH ; BA = (DS) × 16₁₀ ; MA = BA + EA
(DX) ← (MA)
```

Legal register combinations for memory operands (learn the table):

Allowed combinations
[BX + SI], [BX + DI], [BP + SI], [BP + DI]
[SI], [DI], [BX], [BP]
[BX + SI + d8], [BX + DI + d8], [BP + SI + d8], [BP + DI + d8]
[SI + d8], [DI + d8], [BP + d8], [BX + d8]
[BX + SI + d16], [BX + DI + d16], [BP + SI + d16], [BP + DI + d16]
[SI + d16], [DI + d16], [BP + d16], [BX + d16]
d16 (variable offset only — direct addressing)

Only **BX, BP, SI, DI** can go inside [ ]. BX/BP = base (BP→SS), SI/DI = index.

8.8 String Addressing

Employed in **string operations**.

- **EA of source data → SI** register; **EA of destination → DI** register.
- Segment for source = **DS**; segment for destination = **ES**.

```
MOVS BYTE
Source:      EA = (SI)   BA = (DS) × 16₁₀   MA  = BA + EA
Destination: EAE = (DI)  BAE = (ES) × 16₁₀  MAE = BAE + EAE
(MAE) ← (MA)
If DF = 1 → (SI) ← (SI) − 1 ; (DI) ← (DI) − 1
If DF = 0 → (SI) ← (SI) + 1 ; (DI) ← (DI) + 1
```

8.9 Direct I/O Port Addressing (Group III)

An **8-bit port address** is directly specified in the instruction.

```
IN AL, [09H]      PORTaddr = 09H ; (AL) ← (PORT)
```

8.10 Indirect I/O Port Addressing

The instruction specifies the register which holds the port address — in 8086 the **16-bit port address is stored in DX**.

```
OUT [DX], AX      PORTaddr = (DX) ; (PORT) ← (AX)
```

8.11 Relative Addressing (Group IV)

The EA of a program instruction is specified **relative to the Instruction Pointer (IP)** by an **8-bit signed displacement**.

```
JZ 0AH
000AH ← 0AH (sign extend)
If ZF = 1 → EA = (IP) + 000AH ; BA = (CS) × 16₁₀ ; MA = BA + EA  → jump
If ZF = 0 → next instruction is executed
```

8.12 Implied Addressing (Group V)

Instructions using this mode have **no operands** — the instruction itself specifies the data.

```
CLC        ; clears the carry flag to zero  (CF ← 0)
```

9. INSTRUCTION SET OF 8086

8086 supports 6 types of instructions:

1. Data Transfer Instructions
2. Arithmetic Instructions
3. Logical Instructions
4. String Manipulation Instructions
5. Processor Control Instructions
6. Control Transfer Instructions

Notation: ( ) denotes contents of; ← denotes transfer; reg2 = destination, reg1 = source (as used in the slides).

9.1 Data Transfer Instructions

Instructions used to **transfer data/address into registers, memory locations and I/O ports.** Generally **two operands** (source & destination) of the **same size**. Size must be a **byte or a word**; an 8-bit data can move only to an 8-bit register/memory and 16-bit data to a 16-bit register/memory.

Mnemonics: MOV, XCHG, PUSH, POP, IN, OUT

Instruction	Operation
`MOV reg2, reg1`	(reg2) ← (reg1)
`MOV mem, reg1`	(mem) ← (reg1)
`MOV reg2, mem`	(reg2) ← (mem)
`MOV reg, data`	(reg) ← data
`MOV mem, data`	(mem) ← data
`XCHG reg2, reg1`	(reg2) ↔ (reg1)
`XCHG mem, reg1`	(mem) ↔ (reg1)
`PUSH reg16 / mem`	(SP) ← (SP) – 2 ; MA_S = (SS)×16₁₀ + SP ; (MA_S ; MA_S+1) ← (reg16)/(mem)
`POP reg16 / mem`	MA_S = (SS)×16₁₀ + SP ; (reg16)/(mem) ← (MA_S ; MA_S+1) ; (SP) ← (SP) + 2
`IN AL, [DX]`	PORTaddr = (DX) ; (AL) ← (PORT)
`IN AX, [DX]`	PORTaddr = (DX) ; (AX) ← (PORT)
`IN AL, addr8`	(AL) ← (addr8)
`IN AX, addr8`	(AX) ← (addr8)
`OUT [DX], AL`	PORTaddr = (DX) ; (PORT) ← (AL)
`OUT [DX], AX`	PORTaddr = (DX) ; (PORT) ← (AX)
`OUT addr8, AL`	(addr8) ← (AL)
`OUT addr8, AX`	(addr8) ← (AX)

Remember: `MOV` cannot transfer **memory → memory** or **segment register → immediate**.

9.2 Arithmetic Instructions

Mnemonics: ADD, ADC, SUB, SBB, INC, DEC, MUL, DIV, IMUL, IDIV, CMP

Instruction	Operation
`ADD reg2, reg1`	(reg2) ← (reg1) + (reg2)
`ADD reg2, mem`	(reg2) ← (reg2) + (mem)
`ADD mem, reg1`	(mem) ← (mem) + (reg1)
`ADD reg, data`	(reg) ← (reg) + data
`ADD AL, data8`	(AL) ← (AL) + data8
`ADD AX, data16`	(AX) ← (AX) + data16
`ADC …`	Same as ADD **but adds CF**: (reg2) ← (reg1) + (reg2) + CF
`SUB reg2, reg1`	(reg2) ← (reg1) – (reg2)
`SUB reg, data`	(reg) ← (reg) – data
`SBB …`	Same as SUB **but subtracts CF**: (reg) ← (reg) – data – CF
`INC reg8/reg16/mem`	(reg8/16) ← (…)+1 ; (mem) ← (mem)+1
`DEC reg8/reg16/mem`	(reg8/16) ← (…)–1 ; (mem) ← (mem)–1
`MUL reg` (byte)	**(AX) ← (AL) × (reg8)** (result 16-bit)
`MUL reg` (word)	**(DX)(AX) ← (AX) × (reg16)** (result 32-bit)
`MUL mem`	Same pattern with (mem8)/(mem16)
`IMUL reg/mem`	Signed multiply — same register conventions as MUL
`DIV reg` (16÷8)	**(AL) ← quotient, (AH) ← remainder**
`DIV reg` (32÷16)	**(AX) ← quotient, (DX) ← remainder**
`IDIV reg/mem`	Signed divide — same conventions as DIV
`CMP reg2, reg1`	**Modify flags ← (reg2) – (reg1)**

CMP flag results (memorise!):

Condition	CF	ZF	SF
(reg2) > (reg1)	0	0	0
(reg2) < (reg1)	**1**	0	1

Condition	CF	ZF	SF
(reg2) = (reg1)	0	**1**	0

(Same table applies for `CMP reg2, mem`, `CMP mem, reg1`, `CMP reg, data`, `CMP mem, data`, `CMP AL, data8`, `CMP AX, data16`.)

Critical exam point — MUL/DIV operand rules:

- **Byte MUL:** AL × reg8/mem8 → **AX** (16-bit product)
- **Word MUL:** AX × reg16/mem16 → **DX:AX** (32-bit product)
- **Word ÷ Byte:** AX ÷ reg8 → **AL = quotient, AH = remainder**
- **Dword ÷ Word:** DX:AX ÷ reg16 → **AX = quotient, DX = remainder**
- If the quotient cannot fit (divide overflow / divide by zero) → **INT 0 (Type-0 interrupt)** is generated.

9.3 Logical Instructions

Mnemonics: AND, OR, XOR, NOT, TEST, SHR, SHL/SAL, RCR, RCL, ROR, ROL

Instruction	Operation
`AND reg2, reg1`	(reg2) ← (reg2) & (reg1)
`AND reg2, mem`	(reg2) ← (reg2) & (mem)
`AND mem, reg1`	(mem) ← (mem) & (reg1)
`AND reg, data`	(reg) ← (reg) & data
`AND AL, data8` / `AND AX, data16`	(AL) ← (AL) & data8 / (AX) ← (AX) & data16
`OR reg2, reg1`	(reg2) ← (reg2) | (reg1)
`OR reg, data`	(reg) ← (reg) | data
`OR mem, data`	(mem) ← (mem) | data
`OR AL, data8` / `OR AX, data16`	(AL) ← (AL) | data8 / (AX) ← (AX) | data16
`XOR reg2, reg1`	(reg2) ← (reg2) ^ (reg1)
`XOR reg2, mem`	(reg2) ← (reg2) ^ (mem)
`XOR mem, reg1`	(mem) ← (mem) ^ (reg1)
`XOR reg, data`	(reg) ← (reg) ^ data
`XOR AL, data8` / `XOR AX, data16`	(AL) ← (AL) ^ data8 / (AX) ← (AX) ^ data16
`TEST reg2, reg1`	**Modify flags ← (reg2) & (reg1)** — operands unchanged
`TEST mem, reg1`	Modify flags ← (mem) & (reg1)
`TEST reg, data`	Modify flags ← (reg) & data
`TEST AL, data8` / `TEST AX, data16`	Modify flags ← (AL) & data8 / (AX) & data16

TEST vs AND: TEST works the same as AND but **does not change the first operand**. To check whether a number in a register is **even or odd**, TEST can be used **without changing the original number**.

Shift instructions:

Instruction	Operation
`SHR reg/mem, 1` or `SHR reg/mem, CL`	**CF ← B_LSD ; B_n ← B_n+1 ; B_MSD ← 0** (logical shift right; MSB filled with 0)
`SHL reg/mem, 1` (or **SAL**)	**CF ← B_MSD ; B_n+1 ← B_n ; B_LSD ← 0** (logical shift left; LSB filled with 0)

(Diagrams in the slides show the bit movement for 8-bit and 16-bit operands with CF at the end.)

Rotate instructions:

Instruction	Operation
`RCR reg/mem, 1` / `, CL`	**B_n ← B_n-1 ; B_MSD ← CF ; CF ← B_LSD** (rotate through carry **right**)
`RCL`	rotate through carry left (complement of RCR)
`ROL reg/mem, 1` / `, CL`	**B_n-1 ← B_n ; CF ← B_MSD ; B_LSD ← B_MSD** (rotate left, MSB copied to LSB and to CF)
`ROR`	rotate right (complement of ROL)

Count rule: shift/rotate can use either `1` (immediate) or `CL` (variable count) — never any other register.

9.4 String Manipulation Instructions

- **String:** a **sequence of bytes or words**.
- 8086 includes instructions for string **movement, comparison, scan, load and store**.
- **REP prefix** repeats execution of string instructions.
- String mnemonics **end with S / SB / SW** — S = String, SB = String Byte, SW = String Word.
- **Source offset/EA → SI register; destination offset/EA → DI register.**
- **Depending on DF, SI and DI are automatically updated:**
  - **DF = 0 → incremented** by 1 for byte, 2 for word.
  - **DF = 1 → decremented** by 1 for byte, 2 for word.

Mnemonics: REP, MOVS, CMPS, SCAS, LODS, STOS

Prefix	Meaning
`REP`	Repeat while **CX ≠ 0**, and (CX) ← (CX) – 1
`REPZ / REPE`	Repeat CMPS or SCAS while **CX ≠ 0 and ZF = 1**; (CX) ← (CX) – 1 (result should be zero for condition true)
`REPNZ / REPNE`	Repeat CMPS or SCAS while **CX ≠ 0 and ZF = 0**; (CX) ← (CX) – 1 (result should not be zero for condition true)

Example: `repz movsb`

Instruction	Operation
`MOVS (MOVSB, MOVSW)`	MA = (DS)×16_{10} + (SI) ; MAE = (ES)×16_{10} + (DI) ; **(MAE) ← (MA)**. For MOVSW: (MAE;MAE+1) ← (MA;MA+1). DF=0 → SI,DI +1 (or +2 for word); DF=1 → SI,DI –1 (or –2).
`CMPS (CMPSB, CMPSW)`	**Compare two string bytes/words.** Modify flags ← (MA) – (MAE). If (MA) > (MAE): CF=0, ZF=0, SF=0; if <: CF=1, ZF=0, SF=1; if =: CF=0, ZF=1, SF=0. SI/DI updated ±1 (byte) or ±2 (word) as per DF.
`SCAS (SCASB, SCASW)`	**Scan (compare) a string byte/word with the accumulator.** MAE = (ES)×16_{10} + (DI); Modify flags ← **(AL) – (MAE)** (or (AX) – (MAE;MAE+1) for word). DI updated ±1 / ±2 as per DF.
`LODS (LODSB, LODSW)`	**Load string byte into AL / string word into AX.** MA = (DS)×16_{10} + (SI); (AL) ← (MA) [or (AX) ← (MA;MA+1)]; SI updated ±1 / ±2 as per DF.
`STOS (STOSB, STOSW)`	**Store byte from AL / word from AX into string.** MAE = (ES)×16_{10} + (DI); (MAE) ← (AL) [or (MAE;MAE+1) ← (AX)]; DI updated ±1 / ±2 as per DF.

9.5 Processor Control Instructions

Mnemonic	Explanation
`STC`	Set carry: **CF ← 1**
`CLC`	Clear carry: **CF ← 0**
`CMC`	Complement carry: **CF ← CF′**
`STD`	Set direction flag: **DF ← 1**
`CLD`	Clear direction flag: **DF ← 0**
`STI`	Set interrupt enable flag: **IF ← 1**
`CLI`	Clear interrupt enable flag: **IF ← 0**
`NOP`	No operation
`HLT`	Halt after interrupt is set
`WAIT`	Wait for TEST pin active
`ESC opcode mem/reg`	Pass instruction to a coprocessor which shares the address and data bus with the 8086
`LOCK`	Lock bus during next instruction

Note: `STI`/`CLI` control **maskable** interrupts only — NMI can never be masked.

9.6 Control Transfer Instructions

- Transfer control to a specific destination/target instruction.
- **They do NOT affect flags.**

Unconditional transfers:

Mnemonic	Explanation
`CALL reg/mem/disp16`	Call subroutine
`RET`	Return from subroutine
`JMP reg/mem/disp8/disp16`	Unconditional jump

Conditional branch instructions — SIGNED (after CMP, for signed data):

Name	Alternate name
`JE disp8`	Jump if equal
`JZ disp8`	Jump if result is 0
`JNE disp8`	Jump if not equal
`JNZ disp8`	Jump if not zero
`JG disp8`	Jump if greater
`JNLE disp8`	Jump if not less or equal
`JGE disp8`	Jump if greater than or equal
`JNL disp8`	Jump if not less
`JL disp8`	Jump if less than
`JNGE disp8`	Jump if not greater than or equal
`JLE disp8`	Jump if less than or equal
`JNG disp8`	Jump if not greater

Conditional branch instructions — UNSIGNED (after CMP, for unsigned data):

Name	Alternate name
`JE disp8`	Jump if equal
`JZ disp8`	Jump if result is 0
`JNE disp8`	Jump if not equal
`JNZ disp8`	Jump if not zero
`JA disp8`	Jump if above
`JNBE disp8`	Jump if not below or equal
`JAE disp8`	Jump if above or equal
`JNB disp8`	Jump if not below
`JB disp8`	Jump if below
`JNAE disp8`	Jump if not above or equal
`JBE disp8`	Jump if below or equal
`JNA disp8`	Jump if not above

Note (slide): Before these instructions, a **comparison instruction must be used** for comparing two operands.

Conditional branch instructions affecting individual flags:

Mnemonic	Explanation
`JC disp8`	Jump if CF = 1
`JNC disp8`	Jump if CF = 0
`JP disp8`	Jump if PF = 1
`JNP disp8`	Jump if PF = 0
`JO disp8`	Jump if OF = 1
`JNO disp8`	Jump if OF = 0
`JS disp8`	Jump if SF = 1
`JNS disp8`	Jump if SF = 0
`JZ disp8`	Jump if result is zero, i.e. ZF = 1
`JNZ disp8`	Jump if result is not zero, i.e. ZF = 0

Key concept (slide): signed/unsigned conditional branches **check flags**; if the conditions are true, program control is transferred to a new memory location **in the same segment by modifying the content of IP**.

Iteration Control Instructions (execute instructions a number of times):

Instruction	Meaning
`LOOP`	Loop a group of instructions until CX = 0

Instruction	Meaning
LOOPE / LOOPZ	Loop till **ZF = 1 and CX ≠ 0**
LOOPNE / LOOPNZ	Loop till **ZF = 0 and CX ≠ 0**
JCXZ	Jump to the provided address if **CX = 0**

```
MOV CX, 06H
label: ADD AL, BL
       LOOP label          ; decrements CX, loops if CX ≠ 0
```

10. ASSEMBLER DIRECTIVES (Pseudo Instructions)

Definition: Instructions **to the assembler** regarding the program being executed. They **control the generation of machine codes and organisation of the program, but NO machine codes are generated for assembler directives.** Also called **pseudo instructions**.

Used to:

- specify the **start and end of a program**
- **attach values to variables**
- **allocate storage locations** to input/output data
- define **start and end of segments, procedures, macros, etc.**

List of directives in the slides: DB, DW, SEGMENT, ENDS, ASSUME, ORG, END, EVEN, EQU, PROC, FAR, NEAR, ENDP, SHORT, MACRO, ENDM.

10.1 DB (Define Byte)

- Defines a **byte type (8-bit) variable**; reserves specific memory locations for each variable.
- **Range:** 00H–FFH for unsigned; 00H–7FH positive; 80H–FFH negative.
- **General form:** `variable DB value/values`
- **Example:**

```
LIST DB 7FH, 42H, 35H    ; three consecutive memory locations reserved for LIST,
                         ; each holding the specified initial value
```

10.2 DW (Define Word)

- Defines a **word type (16-bit) variable**; reserves **two consecutive** memory locations for each variable.
- **Range:** 0000H–FFFFH unsigned; 0000H–7FFFH positive; 8000H–FFFFH negative.
- **General form:** `variable DW value/values`
- **Example:**

```
ALIST DW 6512H, 0F251H, 0CDE2H   ; six consecutive memory locations reserved
```

10.3 Remaining Directives

Directive	Meaning
SEGMENT	Indicates the **beginning** of a code/data/stack segment. General form: `Segnam SEGMENT … Segnam ENDS`
ENDS	Indicates the **end** of a code/data/stack segment
ASSUME	Informs the assembler which segment name should be used for a specific segment register. `ASSUME segreg : segnam, …` — e.g. `ASSUME CS: ACODE, DS: ADATA` tells the assembler that program instructions are stored in segment **ACODE** and data in **ADATA**
ORG	**Origin** — assigns the **starting address (EA)** for a program/data segment. `ORG 1000H` → statements following it are stored from effective address 1000H
END	Terminates a program; **statements after END are ignored**
EVEN	Informs the assembler to store program/data segment starting from an **even address**
EQU	**Equate** — attaches a value to a variable. `LOOP EQU 10FEH` → value of variable LOOP is 10FEH
PROC / ENDP	Beginning / end of a **procedure**
NEAR	**Intrasegment** call (within the same code segment)
FAR	**Intersegment** call (different code segment)
SHORT	Reserves one memory location for an **8-bit signed displacement** in jump instructions: `JMP SHORT AHEAD`
MACRO / ENDM	Beginning / end of a **macro**: `macroname MACRO [Arg1, Arg2 …] … ENDM`

Worked ORG/EVEN example (from slides):

```
_SDATA SEGMENT
       ORG 1200H
       A DB 4CH
       EVEN
       B DW 1052H
_SDATA ENDS
```

→ Effective address of **A = 1200H**; B occupies **1202H and 1203H**.

PROC examples:

```
ADD64 PROC NEAR        ; assembler codes CALL/RET as NEAR call & return
   …
   RET
ADD64 ENDP

CONVERT PROC FAR       ; assembler codes CALL/RET as FAR call & return
   …
   RET
CONVERT ENDP
```

10.4 Assembler Operators

Operator	Meaning
LENGTH	Tells the assembler to determine the **number of elements** in a named data item (string/array), e.g. `MOV CX, LENGTH STRING1` loads the count of elements in STRING1 into CX. For a byte string → number of bytes; word string → number of words. Also: byte length of a label — `MOV CX, LENGTH ARRAY`.
OFFSET	Returns the **16-bit offset address of a label**; the assembler replaces "OFFSET label" by the computed offset. e.g. `MOV SI, OFFSET LIST`

Operator	Meaning
LEA	**Load Effective Address** — loads the address of a variable. e.g. `LEA AX, TEST` for `TEST DB 23H, 40H, 44H`

11. PROCEDURES, MACROS AND MODULAR PROGRAMMING

11.1 Procedure

A **procedure is a group of instructions that usually performs one task.** It is a **reusable section of software** stored in memory **once**, but can be used as often as necessary.

Two types:

- **NEAR procedure:** written (defined) in the **same code segment** that calls it → **only IP changes**.
- **FAR procedure:** written in a **different code segment** than the calling segment → **both IP and CS change**.

Directives: PROC identifies the **start** of a procedure (followed by a name and NEAR/FAR); **ENDP** with the procedure name marks the **end**. PROC and ENDP **bracket** a procedure.

11.2 CALL and RET

CALL transfers execution to a procedure. It performs two operations:

1. **Stores the address of the instruction after CALL on the stack.**
2. **Changes IP** (Near call) or **IP and CS** (Far call).

Near Call (Intrasegment): 8086 **decrements SP by 2** and copies **IP** onto the stack; then copies the **address of the first instruction of the called procedure** into IP.

Far Call (Intersegment): 8086 **decrements SP by 2** and pushes **CS**; **decrements SP by 2 again** and pushes **IP**; then **loads CS** with the base address of the segment containing the procedure and **IP** with the address of its first instruction.

11.3 Procedure vs Macro (COMPARISON TABLE — high-yield)

Procedure	Macro
Used for **large** groups of instructions to be repeated	Used for **small** groups of instructions to be repeated
Object code is generated **only once** in memory	Object code is generated **every time the macro is called**
CALL & RET instructions are used to call and return	Macro is called **just by writing its name**
Length of the object file is **less**	Object file becomes **lengthy**
PROC & ENDP directives are used for defining a procedure	**MACRO and ENDM** directives are used for defining a macro
More time is required for its execution	**Less time** is required for its execution
Defined as: `Procedure_name PROC … Procedure_name ENDP`	Defined as: `MACRO-name MACRO [ARGUMENT, … ARGUMENT N] … ENDM`
Example: `Addition PROC near … Addition ENDP`	Example: `Display MACRO msg … ENDM`

11.4 Modular Programming

Modular programming is a software design technique that emphasises separating the functionality of a program into independent, interchangeable modules, such that each contains everything necessary to execute **only one aspect** of the desired functionality.

- A **module interface** expresses the elements that are **provided and required** by the module; elements in the interface are detectable by other modules.
- The **implementation** contains the working code corresponding to the elements declared in the interface.
- Closely related to **structured programming** and **object-oriented programming** — all share the goal of facilitating construction of large software by **decomposition into smaller pieces** (all originated around the 1960s).
- **New usage:** "modular programming" = **high-level decomposition of the code of an entire program into pieces**; structured programming = low-level code use of structured control flow; OOP = data use of objects.

12. 8086 INTERRUPTS

12.1 Sources of Interrupts (3 types)

1. An **external signal** applied to the **NMI or INTR** input pin → **(Hardware interrupt)**
2. Execution of an **INTn (n = 00H–FFH) instruction** → **(Software interrupt)**
3. **Interrupt caused by some error condition** produced in the 8086 instruction execution process — (e.g. **divide by zero, overflow errors**)

Interrupt classification chart (from slides):

```
8086 Interrupts
├── Hardware Interrupts
│     ├── Maskable Interrupt (INTR)
│     │       → The programmer can choose to mask specific interrupts
│     │         and re-enable them later
│     └── Non-maskable Interrupt (NMI)
│           → The programmer CANNOT control when a non-maskable
│             interrupt is served
│           → The processor has to stop the main program to execute
│             the NMI Service Routine
└── Software Interrupts (INT n)
      → 256 types of software interrupts, INT 00 to INT FF
```

INTR (pin 18) is where maskable hardware interrupts are applied; NMI (pin 17) is non-maskable.

Uses of Non-Maskable Interrupts:

- Used during **power failure**
- Used during **critical response times**
- Used during **non-recoverable hardware errors**
- Used as **watchdog interrupt**
- Used during **memory parity errors**

12.2 Interrupt Processing — Steps performed by the 8086

If an interrupt has been requested, the 8086 processes it by performing the following series of steps:

1. **Pushes the content of the flag register onto the stack** to preserve the status of IF and TF flags, by **decrementing SP by 2**.
2. **Disables the INTR interrupt by clearing IF** in the flag register.

3. **Resets TF** in the flag register, to disable the single-step (trap) interrupt.
4. **Pushes the content of CS** onto the stack by decrementing SP by 2.
5. **Pushes the content of IP** onto the stack by decrementing SP by 2.
6. **Performs an indirect far jump to the start of the Interrupt Service Routine (ISR)** corresponding to the received interrupt.

Flow (slide version):

```
Main Program → Executes the Interrupt instruction → Jumps to the Interrupt Vector Table
→ Takes the CS and IP in the Vector Table → Pushes the existing CS and IP on the Stack
→ Loads the new CS and IP → Jumps to the ISR → Executes ISR
→ Comes back and continues the Main Program
```

Hardware action diagram:

```
Interrupt → Push flag register → Clear IF and TF → Push CS and IP → Load CS and IP
   → [Interrupt Service Routine (ISR) … ends with IRET]
   → Pop IP and CS → Pop flags register → main program continues
```

Remember the order: FLAGS → CS → IP pushed; IRET pops IP → CS → FLAGS.

12.3 Interrupt Vector Table (IVT)

- Located at the **bottom of memory**: addresses **00000H – 003FFH** (1 KB).
- Holds **256 interrupt vectors**, each vector = **4 bytes** (2 bytes for IP + 2 bytes for CS).
- **For a vector of type n, the ISR address is stored at:**

Offset = Type number × 4

Example: INT 02H → Offset = 02 × 4 = 08 = 00008H

Layout of one vector (Type 0 example):

Memory address	Content
00000H	IP low byte
00001H	IP high byte
00002H	CS low byte
00003H	CS high byte

IVT layout (memory address in hex):

Address	Vector contents
00000H–00003H	Type 00H — Divide by Zero
00004H–00007H	Type 01H — Trap/Single step
00008H–0000BH	Type 02H — NMI
0000CH–0000FH	Type 03H — Break point
00010H–00013H	Type 04H — Overflow
00014H–0007FH	Type 05H — Reserved
00080H–00083H	Type 20H — Available
00084H–00087H	Type 21H — Available
…	…
003FCH–003FFH	Type FFH — Available

Order inside each 4-byte vector: lower 2 bytes = IP, upper 2 bytes = CS.

12.4 The 256 Interrupts are Divided into 3 Groups

Group	Types	Purpose
1	**Type 00 to Type 04**	Used for **fixed operations** → called **dedicated interrupts** (5 total)
2	**Type 05 to Type 31**	**Not used by 8086; reserved for higher processors** like 80286, 80386 etc. (27 reserved)
3	**Type 32 to Type 255**	**Available for the user → user-defined interrupts**; can be H/W interrupts activated through the INTR line or S/W interrupts (224 available)

12.5 Dedicated Interrupts — Types 0 to 4 (MUST MEMORISE)

Type	Name	Description
Type 0	**Divide by Zero Error Interrupt**	Occurs when the **quotient is too large to fit in AL/AX**, or on **divide by zero**. (Example: `MOV AL,05H / MOV DL,00H / DIV DL / HLT` → "Attempted to divide by zero" error.)
Type 1	**Single step or Trap Interrupt**	Used for **executing the program in single-step mode** by setting the **trap flag**.
Type 2	**Non-Maskable Interrupt (NMI)**	Used for executing the **ISR of the NMI pin** (**positive edge signal**); **NMI cannot be masked by software.**
Type 3	**One-byte INT instruction interrupt**	Used for providing **break points** in the program.
Type 4	**Overflow Interrupt**	Used to handle **any overflow error after signed arithmetic** (of = 1).

13. EXAMPLE PROGRAMS (write these in the exam — they carry full marks)

13.1 Procedure Example Program (with data + stack + code segments)

```
ASSUME CS:CODE, DS:DATA, SS:STACK_SEG

DATA SEGMENT
  NUM1    DB 50H
  NUM2    DB 20H
  ADD_RES DB ?
  SUB_RES DB ?
DATA ENDS

STACK_SEG SEGMENT
  DW 40 DUP(0)          ; stack of 40 words, all initialized to zero
  TOS LABEL WORD
STACK_SEG ENDS

CODE SEGMENT
START: MOV AX, DATA          ; initialize data segment
       MOV DS, AX
       MOV AX, STACK_SEG     ; initialize stack segment
       MOV SS, AX
       MOV SP, OFFSET TOS    ; initialize stack pointer to TOS
       CALL ADDITION
       CALL SUBTRACTION
       MOV AH, 4CH           ; return to DOS
       INT 21H

ADDITION PROC NEAR
       MOV AL, NUM1
       MOV BL, NUM2
       ADD AL, BL
       MOV ADD_RES, AL
       RET
ADDITION ENDP

SUBTRACTION PROC
       MOV AL, NUM1
       MOV BL, NUM2
       SUB AL, BL
       MOV SUB_RES, AL
       RET
SUBTRACTION ENDP

CODE ENDS
END START
```

Key points to write as comments: `MOV AX, DATA` → `MOV DS, AX` because a **segment register cannot be loaded with an immediate** — must go through a general-purpose register.

13.2 Macro Example Program

```
ASSUME CS:CODE, DS:DATA

DATA SEGMENT
  NUM1 DW 1000H
  NUM2 DW 2000H
  RES  DW ?
DATA ENDS

CODE SEGMENT
ADDITION MACRO NO1, NO2, RESULT
         MOV AX, NO1
         MOV BX, NO2
         ADD AX, BX
         MOV RESULT, AX
         ENDM

START: MOV AX, DATA       ; initialize data segment
       MOV DS, AX
       ADDITION NUM1, NUM2, RES     ; macro call — just write its name
       MOV AH, 4CH
       INT 21H
CODE ENDS
END START
```

13.3 Divide-by-Zero Demonstration (Emu8086)

```
MOV AL, 05H
MOV DL, 00H
DIV DL        ; → divide by zero → Type 0 interrupt
HLT
```

13.4 Overflow Demonstration

A signed arithmetic overflow (e.g. `INT32` overflow in the slides' example) triggers the **Type-4 Overflow interrupt** handling path — the same principle as `INT0`/OF flag handling.

14. COMMON EXAM QUESTIONS ON THIS MODULE (with 1-line answers)

Question	Answer skeleton
Draw and explain the architecture of 8086.	BIU (segment registers CS/DS/SS/ES, IP, dedicated 20-bit adder, 6-byte queue) + EU (ALU, general registers AX/BX/CX/DX, SP/BP, SI/DI, flag register) — explain **pipelining** (fetch & execute simultaneously).
Explain the pin diagram of 8086 / difference between minimum and maximum mode pins.	Use §4 tables: AD0–AD15, A16/S3–A19/S6, BHE/S7, MN/MX, RD, READY, RESET, CLK, INTR, NMI, TEST; min-mode ALE, DEN, DT/R, M/IO, WR, INTA, HOLD, HLDA; max-mode S0–S2, QS0–QS1, LOCK, RQ/GT0–RQ/GT1 + MN/MX = 1 min / 0 max.
How is the physical address calculated?	PA = (Segment × 16_{10}) + Offset; example 89AB:F012 = 98AC2H.
List and explain the addressing modes with examples.	All 12 modes with the worked EA/BA/MA examples of §8.
Explain the flag register.	Bits: CF(0), PF(2), AF(4), ZF(6), SF(7), TF(8), IF(9), DF(10), OF(11).
Difference between procedures and macros.	Use the 7-row comparison table of §11.3.

Question	Answer skeleton
Difference between NEAR and FAR call.	NEAR: only IP pushed/changed; FAR: CS then IP pushed, both changed.
What is the IVT? How is the vector address calculated?	00000H–003FFH, 256 vectors × 4 bytes; Offset = Type × 4; lower 2 bytes = IP, upper 2 = CS.
Explain the interrupt processing steps of 8086.	Push flags → clear IF → reset TF → push CS → push IP → indirect far jump to ISR (§12.2).
Explain types of interrupts in 8086.	Hardware (INTR/NMI) + Software (INT n) + error conditions; dedicated types 0–4, reserved 5–31, available 32–255.
Explain string instructions.	MOVS/CMPS/SCAS/LODS/STOS + REP/REPZ/REPNZ, SI/DI with DS/ES, DF effect.
Explain assembler directives.	DB, DW, SEGMENT, ENDS, ASSUME, ORG, END, EVEN, EQU, PROC, ENDP, FAR, NEAR, SHORT, MACRO, ENDM.
Explain MUL/DIV instructions.	Byte: AL×operand→AX; Word: AX×operand→DX:AX. DIV: AX÷r8 → AL quotient, AH remainder; DX:AX÷r16 → AX quotient, DX remainder.

15. LAST-MINUTE REVISION CRAM SHEET

Numbers to remember:

Item	Value
8086 released / word size	**1978 / 16-bit**
Transistors	**29,000**
Package / supply / clock duty cycle	**40-pin DIP / +5 V / 33 % (asymmetric)**
Address bus / memory	**20-bit / 1 MB (2^{20})**
Data bus	16-bit
Segment size	**64 KB** (4 segments active = 256 KB)
Instruction queue	**6 bytes FIFO**
Physical address formula	**(Seg × 16_{10}) + Offset**
IVT location & size	**00000H–003FFH, 1 KB, 256 vectors × 4 bytes**
Vector address	**Type number × 4**
Interrupt groups	**05 dedicated (0–4), 27 reserved (5–31), 224 available (32–255)**
RESET must be high for	**at least 4 clock cycles**

Flags: `CF` bit 0, `PF` 2, `AF` 4, `ZF` 6, `SF` 7, `TF` 8, `IF` 9, `DF` 10, `OF` 11.

Modes: `MN/MX = 1` → **Minimum** (no coprocessor); `MN/MX = 0` → **Maximum** (multiprocessor, needs 8288).

Chips: **8284** = clock generator, **8282** = latch (address demux with ALE), **8286** = transceiver (data bus), **8288** = bus controller (max mode).

If you can explain only two things, explain: (a) BIU/EU + pipelining, and (b) the 12 addressing modes with one example each.

Prepared from the Module 2 lecture slides (ECE3004 – MPMC). Diagrams referred to in the slides should be reproduced alongside these notes: BIU/EU block diagram, register/flag layout, segment-register memory map, ALE/8282/8286 demultiplexing circuit, shift/rotate bit diagrams, IVT map, and interrupt processing flowchart.